

Arquivos e Manipulação de Arquivos Texto

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz
marcosleipnitz@ic.ufrj.br

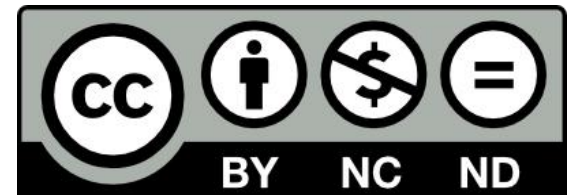


UFRJ

UNIVERSIDADE FEDERAL
DO RIO DE JANEIRO

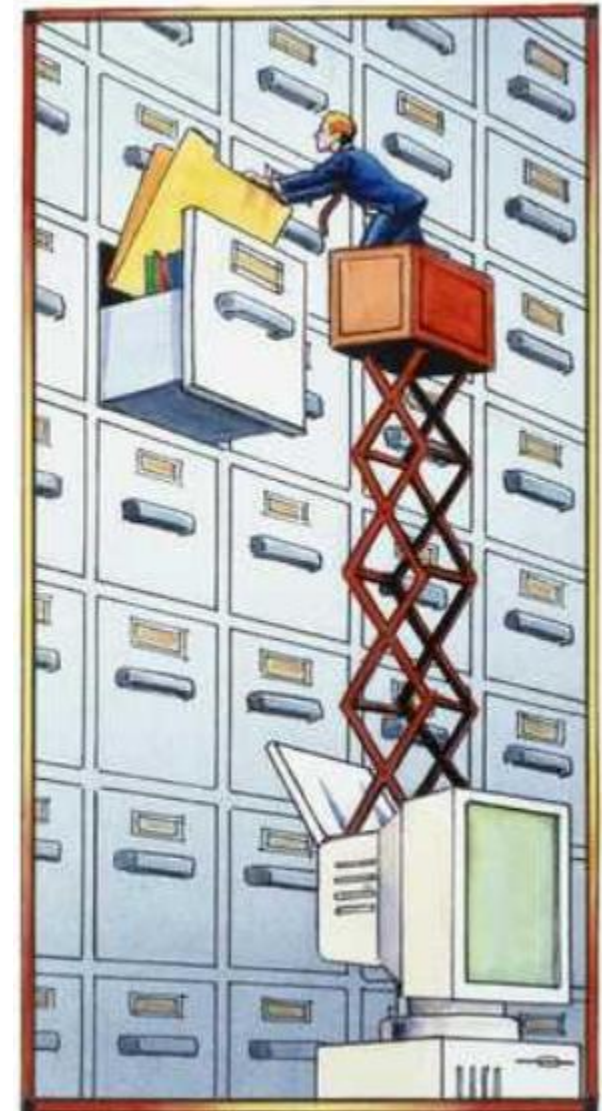


INSTITUTO DE
COMPUTAÇÃO
UFRJ

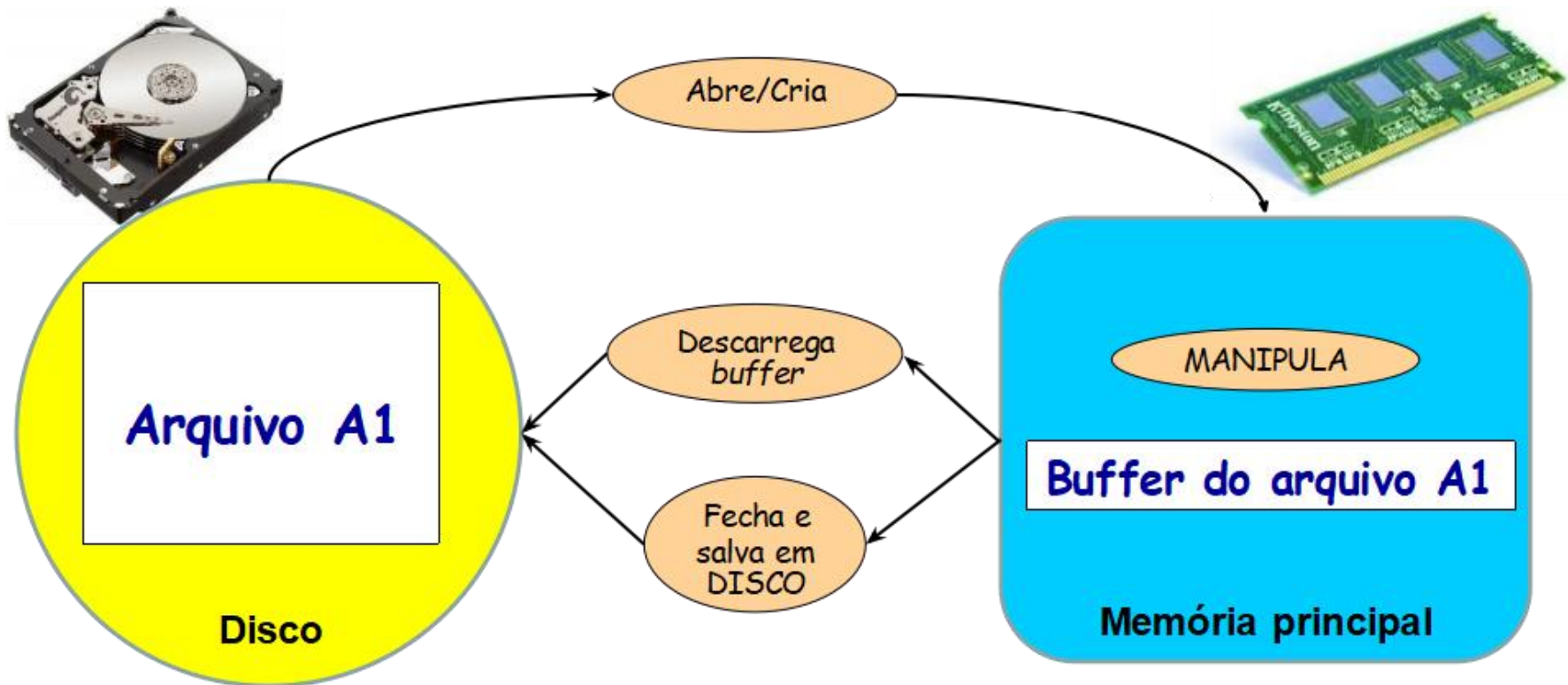


- Arquivos
 - Fluxos de dados
 - Manipulação de arquivos
 - Arquivos texto
- Exemplos

- Repositórios **permanentes** de dados (informações) que possibilitam a **comunicação dos programas** com o ambiente.
- Usualmente armazenados em dispositivos de memória auxiliar.
 - HD, SSD, etc
- A alteração de um arquivo durante a execução de um programa torna **permanente** o processamento realizado.
 - Persistência de informações.
- Em arquivos é possível armazenar uma quantidade muito maior de informações do que na **memória principal**.



Arquivos



- Em C, a manipulação de informações em memória secundária envolve duas noções importantes:

➤ **Arquivo**

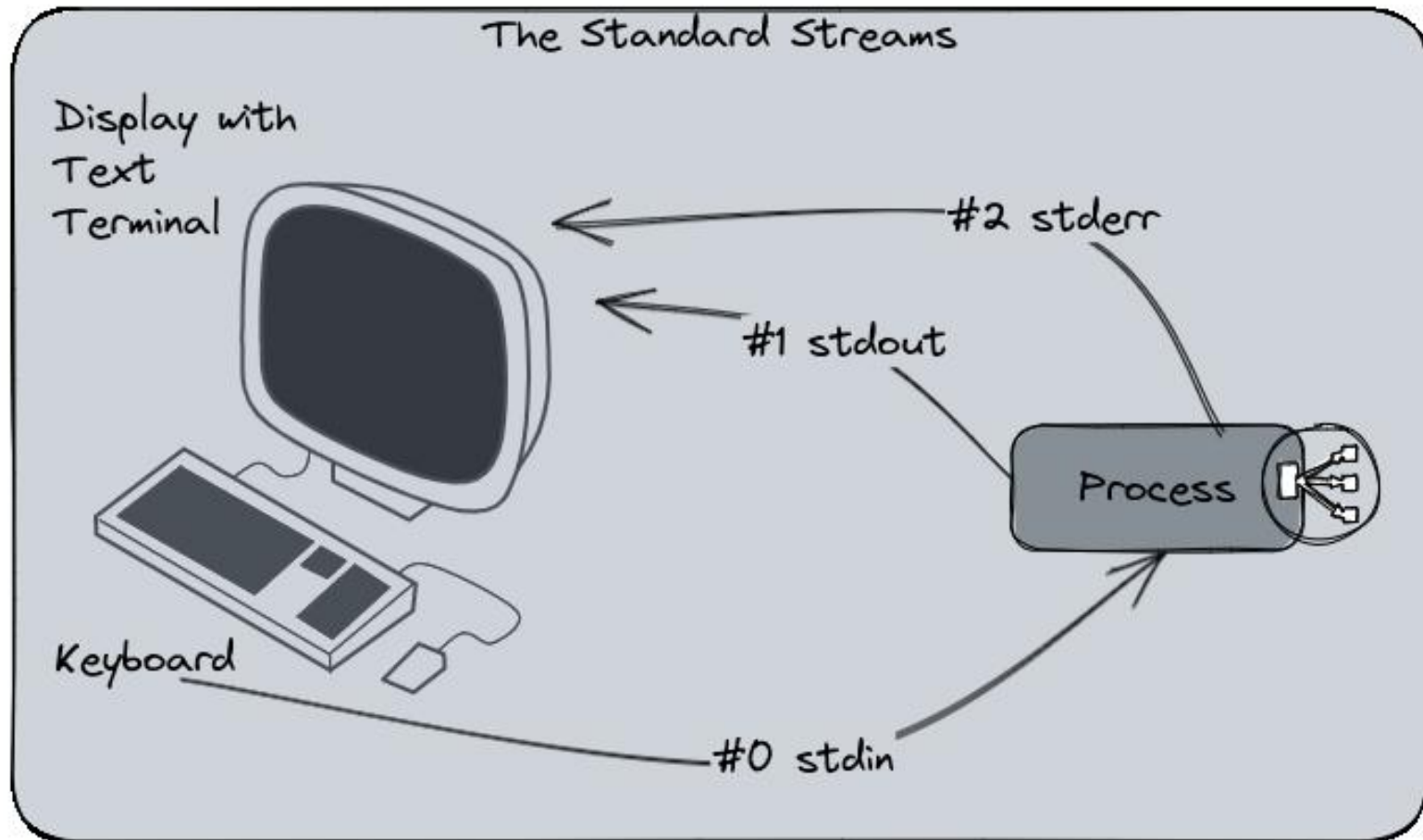
- **Fonte de dados externa:** Pode ser muitas coisas, como um arquivo em um disco, uma impressora, ou outros dispositivos para os quais podemos enviar ou receber informações.



- Em C, a manipulação de informações em memória secundária envolve duas noções importantes:
 - **Fluxo de dados**: Para isolar os programadores dos problemas de manipular os vários tipos de dispositivos de armazenamento e seus diferentes formatos, a linguagem C utiliza o conceito de fluxo de dados (stream)
 - É uma **abstração lógica** do fluxo de dados com um arquivo.
 - Permite trabalhar com múltiplos dispositivos, através de uma mesma interface (um mesmo conjunto de operações padrão).
 - Streams estão associadas a áreas temporárias da memória principal denominadas **buffers**.
 - Operações sobre streams são realizadas sobre o **buffer**, e não propriamente no disco (ou outro dispositivo).
- Uma **fluxo** é **associado** a um **arquivo** através de uma operação de **abertura**.
- Toda a **entrada** e **saída** de dados é realizada através de **fluxos de texto ou binários**.

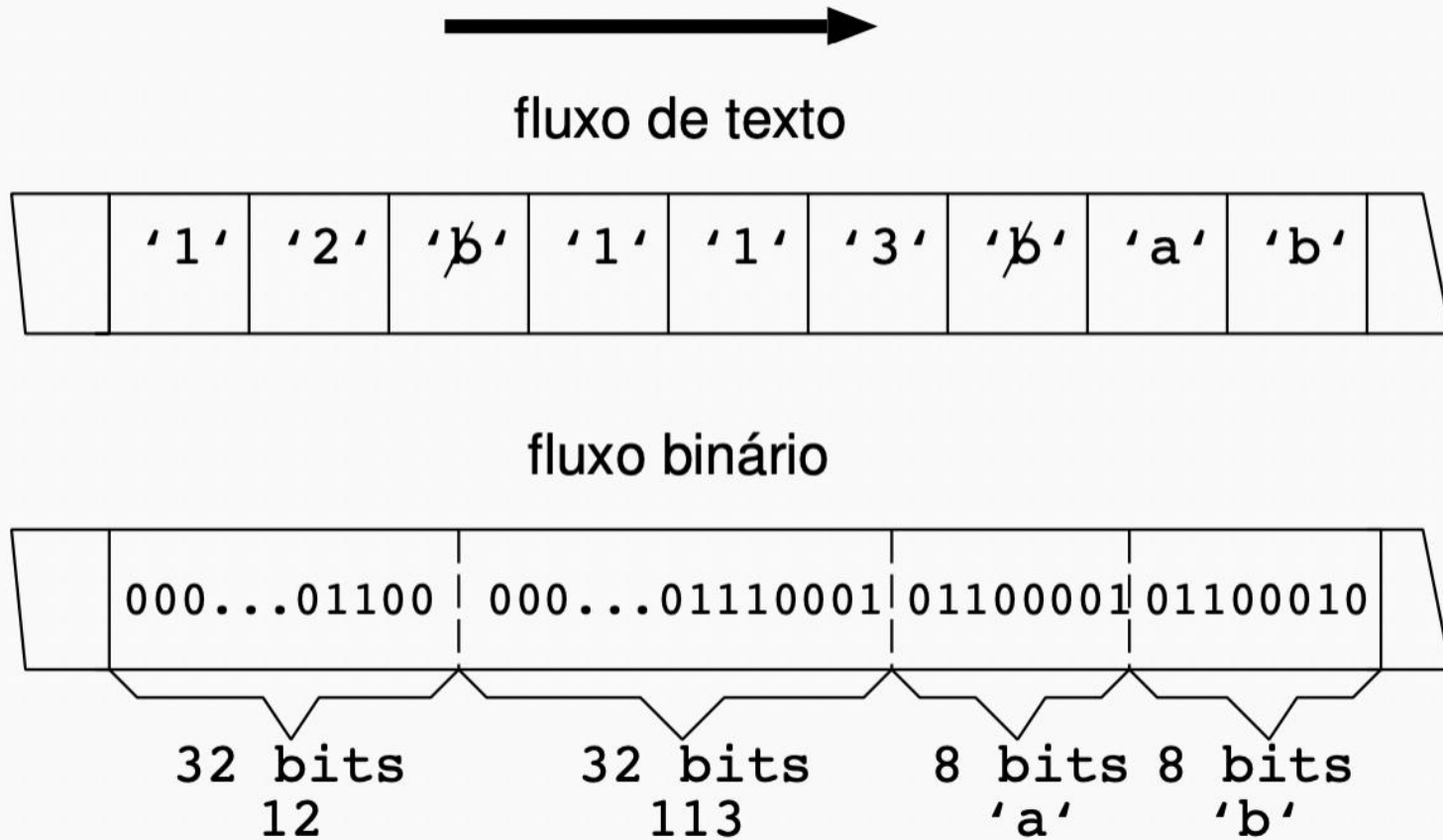
- Um **fluxo de texto** é composto por uma sequência de caracteres, que pode dividida em linhas terminadas por um caractere de final de linha.
- Fluxos de dados padrão:
 - **stdin**, para entrada de dados, normalmente associado ao teclado;
 - **stdout** para saída de dados, normalmente associado ao vídeo.
- Ao iniciar todo programa em C é automaticamente associado a estes dois fluxos de dados.
- A definição de que periféricos estarão associados a estes fluxos depende do sistema operacional

Fluxos Padrão

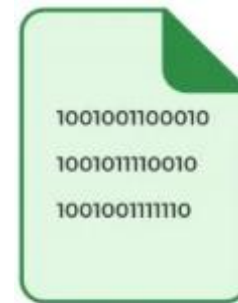
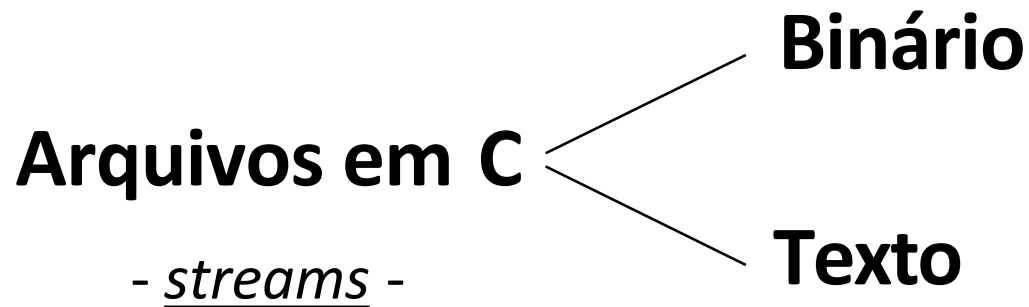


- Um **fluxo binário** é composto por uma sequência de bytes lidos, sem tradução, diretamente do dispositivo externo
- Existe uma correspondência um para um entre os dados do dispositivo e os que estão no fluxo que o programa manipula
- No fluxo de texto os dados são armazenados como caracteres sem conversão para representação binária
- No fluxo binário cada número inteiro ocupa 32 bits e é armazenado na forma binária

Fluxos de Dados



Conjunto de **DADOS** armazenados em **memória secundária**
(de forma permanente)



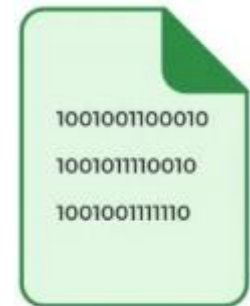
file.bin



file.txt

Tipos de Arquivos em C

- **Binário:** composto por **uma sequência** (fluxo binário) **de bytes**, lidos **diretamente do dispositivo externo**. Não ocorre nenhuma tradução direta e existe uma correspondência um para um entre os dados do dispositivo e os que estão no fluxo.
- **Texto:** composto de uma sequência de bytes correspondendo a uma sequência de caracteres ASCII, incluindo **letras**, **dígitos** e **caracteres especiais**, bem como os separadores **branco**, **tab** e **new line** (indicando final de linha e, dependendo do ambiente, utilizando para isso 1 ou 2 caracteres ASCII). Tais sequências podem ou não serem divididas em linhas terminadas por um caractere de final de linha.



file.bin



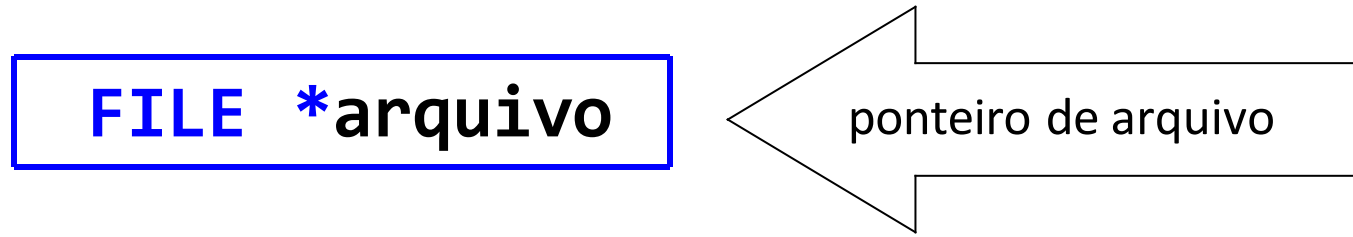
file.txt

Operações em Arquivos

- **Abrir** um arquivos
- **Fechar** um arquivo
- **Renomear** um arquivo
- **Remover** um arquivo
- **Leitura** e **escrita** de um caractere ou byte
- Procurar saber se o **fim do arquivo** foi atingido
- **Posicionar** o arquivo em um ponto determinado
- Entre outras

Funções em C

Função	Descrição
<code>fopen()</code>	Abre um arquivo
<code>fclose()</code>	Fecha um arquivo
<code>fputc()</code>	Escreve um caractere em um arquivo
<code>getc()</code> , <code>fgetc()</code>	Lê um caractere de um arquivo
<code>fprintf()</code>	Equivalente a <code>printf()</code>
<code>sscanf()</code>	Equivalente a <code>scanf()</code> ; Lê de uma cadeia de caracteres
<code>fscanf()</code>	Equivalente a <code>scanf()</code>
<code>fseek()</code>	Posiciona o arquivo em um ponto específico
<code>rewind()</code>	Posiciona o arquivo no início
<code>feof()</code>	Retorna verdade se chegou ao fim do arquivo
<code>ferror()</code>	Verifica a ocorrência de um erro
<code>fflush()</code>	Descarrega o <i>buffer</i> associado ao arquivo
<code>fread()</code>	Leitura de dados no modo binário
<code>fwrite()</code>	Escrita de dados no modo binário



- Os streams de arquivos são representados por uma **estrutura** chamada **FILE**, definida em stdio.h.
- Os programa que manipulam estas streams usam então um **ponteiro** para esta estrutura (**FILE ***).

Não manipulamos FILE diretamente!

Estrutura FILE

```
c Copy code

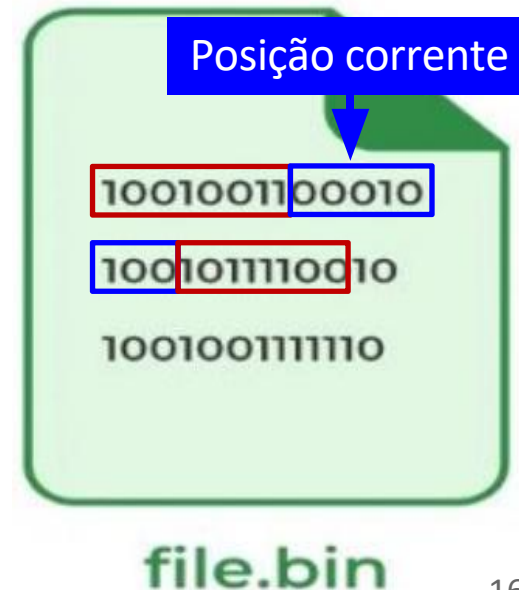
typedef struct _IO_FILE {
    // Campos relacionados à posição no arquivo
    long int posicao; // Posição atual do "file pointer"
    // ...

    // Campos relacionados ao modo de abertura do arquivo
    char modo_abertura[4]; // Modo de abertura ("r", "w", "a", etc.)
    // ...

    // Outros campos relevantes, como o buffer de entrada/saída, indicadores
    // ...
} FILE;
```

f->posicao armazena a “posição corrente”
(onde está o próximo byte a ser lido/escrito
dentro deste arquivo)

Exemplo de código
interno hipotético da
estrutura FILE via
ChatGPT



- A estrutura FILE é uma estrutura “opaca”, o que significa que seus **detalhes internos não são expostos ao programador**;
- Isso é intencional para garantir a **portabilidade e independência** do sistema operacional;
- Os detalhes de implementação da estrutura FILE são definidos pela biblioteca padrão C e **não devem** ser acessados diretamente pelo programador, podendo levar a comportamentos indefinidos;
- Interações de arquivos devem ser feitas apenas por meio de ponteiros FILE* **retornados por funções**;
- Esta é uma característica de **boa prática** baseada no **encapsulamento** utilizando **interfaces**

Esquema de utilização de arquivos

A utilização de arquivo em C envolve os seguintes passos:

1. **Declaração de um ponteiro do tipo FILE, que representa e controla o stream do arquivo:**

```
FILE *point_arq;
```

2. **Abertura do arquivo, onde o ponteiro do tipo FILE declarado é associado ao arquivo externo através de**

```
point_arq = fopen (<nome do arquivo>, <tipo de acesso>);
```

3. **Operações com arquivo, através de funções de manipulação de arquivos (aplicadas ao ponteiro).**

4. **Fechamento do arquivo, através de**

```
fclose (point_arq);
```

Declaração de Arquivos

- Para que um arquivo possa ser usado pelo programa é necessário que seja declarado um **ponteiro** para uma **estrutura do tipo FILE**.
 - Cada arquivo a ser usado simultaneamente pelo programa é associado a um **apontador para FILE** específico.

```
// Programa que utiliza arquivos
#include <stdio.h> // e demais bibliotecas
...
int main( )
{
    FILE *arq; //declaração de um “ponteiro de arquivo”
    FILE *arq1, *arq2; // outro exemplo com 2 ponteiros
    .....
}
```

Este programa é capaz de lidar com 3 arquivos

```
point_arq = fopen(<nome do arquivo>, <tipo de acesso>);
```

- **Abre/cria/recria** um arquivo, com o nome completo do arquivo físico e o modo de abertura desejado, passados como parâmetros do tipo *string*.
- A função retorna, no **ponteiro do tipo FILE** associado ao arquivo, o endereço inicial deste arquivo ou, se a função não tiver sido bem sucedida, o valor **NULL**.
- Após abertura, **verificar SEMPRE** se não houve erro, testando se o retorno é **diferente de NULL**.

Abertura de Arquivos

Há 2 maneiras principais de testar se abertura foi realizada com sucesso:

```
#include <stdio.h> // inclui as funções com arquivos

int main( )
{
    FILE *arq1, *arq2; // declaração de apontadores de arquivo
    arq1 = fopen("c:\\exemplo1.txt", "r"); // abertura para leitura
    if (arq1 == NULL) // verifica se funcionou: retorno NULL indica erro
        printf("Erro na abertura do arquivo 1.");
    else{
        // abertura para leitura, junto à análise do resultado da operação:
        if (!(arq2 = fopen("c:\\exemplo2.bin", "rb"))) // not(null) = true
            printf("Erro na abertura do arquivo 2.");
        ....
    }
}
```

Arquivo texto

Arquivo binário

Abertura de Arquivos

Há 2 maneiras principais de testar se abertura foi realizada com sucesso:

```
#include <stdio.h> // inclui as funções com arquivos

int main( )
{
    FILE *arq1, *arq2; // declaração de apontadores de arquivo
    arq1 = fopen("c:\\exemplo1.txt", "r"); // abertura para leitura
    if (arq1 == NULL) // verifica se funcionou: retorno NULL indica erro
        printf("Erro na abertura do arquivo 1.");
    else{
        // abertura para leitura, junto à análise do resultado da operação:
        if (!(arq2 = fopen("c:\\exemplo2.bin", "rb"))) // not(null) = true
            printf("Erro na abertura do arquivo 2.");
        ....
    }
}
```

Forma mais “legível”

Forma mais “compacta”

Fechamento e Descarga

```
fclose(point_arq );
```

- **Fecha** um *stream* de arquivo
- Obs.: Ao fechar o *stream* do arquivo, **todos os dados** que ainda estavam em *buffer* **são salvos em disco**.

Usado para forçar a escrita dos dados naquele exato momento

```
fflush(point_arq );
```

- **Descarrega** o conteúdo em *buffer* para o arquivo, sem fechar o *stream*.
- Efetiva operações de funções de gravação (como **fwrite**) que ainda não tenham sido efetivadas (*buffer* -> HDD/SSD).

```
feof(point_arq)
```

- Verifica se um arquivo chegou ao fim.
- Retorna verdadeiro (não-zero) se o final for atingido ou falso (zero), caso contrário.
- **ATENÇÃO:** O `feof` só é detectado após a leitura (mal sucedida) do final do arquivo, quando o valor retornado pela leitura for diferente do especificado
 - Neste caso, esta leitura **deve ser ignorada!**

```
rewind(point_arq)
```

- Coloca a **posição corrente** do arquivo para o **início do arquivo**.
- Usada quando, após a abertura do arquivo, tiverem sido executadas funções de leitura ou gravação, que tenham deslocado a posição corrente para depois do início do arquivo.

Posicionamento Aleatório

fseek(point_arq,numbytesdesloc,origem)

- **Move** a posição corrente de leitura/escrita para a posição calculada:
 - **origem + numbytesdesloc.**
- Os valores possíveis, definidos, para **origem** do deslocamento são:
 - **SEEK_SET** ou **0** = início do arquivo
 - **SEEK_CUR** ou **1** = ponto corrente no arquivo
 - **SEEK_END** ou **2** = fim do arquivo
- Usar apenas quando quiser alterar o acesso sequencial ao arquivo.

```
...  
FILE *arq;  
char c;  
arq = fopen(...);  
/*coloca a posição para o byte 34 do arquivo*/  
fseek(arq,33,SEEK_SET);  
c = getc(arq); // lê o caractere 34  
...  
...
```

ftell(point_arq)

- **Indica** a posição corrente no arquivo.
- Posição dada em bytes a partir do início do arquivo.

```
...  
FILE *arq;  
arq = fopen(...);  
...  
  
/*posição corrente = início*/  
printf("\nposicao atual em bytes antes leitura%d\n", ftell(arq));  
if (fread(&buffer, sizeof(ATLETA), 1, arq) == 1)  
    printf("\nposicao atual em bytes apos leitura%d\n", ftell(arq));  
...  
...
```

```
...                               binário (bin, png, jpg, data, etc.)
FILE *arq;
arq = fopen("exemplo.bin", "rb");
// se ocorrer erro, arq recebe NULL
if (!arq){
    printf("Erro na abertura do arquivo.");
    /* na sequência encerra a execução do programa */
}else{
    /* efetua operações com arquivo */
    fclose(arq); /* fecha o arquivo */
}
...
```

Exemplo

```
...  
FILE *arq;  
...  
while(!feof(arq)) // enquanto não for o fim, faça  
{  
    if (fread (&varbuffer,numbytes,[quant], point_arq) == [quant])  
    { // leitura correta  
        ...  
    }  
    // else // leu o final do arquivo  
}  
...
```



Caso a **continuidade de leitura** seja **condicionada** pelo **feof**, é importante **sempre** testar dentro do laço se a **leitura** foi um **sucesso**.

Outras funções que funcionam em arquivos texto da mesma forma que em arquivos binários:

- **ferror**: verifica se ocorreu um erro em uma operação de arquivo.
- **rename**: renomeia um arquivo no sistema de arquivos.
- **remove**: exclui um arquivo do sistema de arquivos permanentemente.

Verifica Ocorrência de Erro

`ferror(point_arq)`

- Indica se a **última operação** com um arquivo produziu um **erro**.
- Retorna **verdadeiro** (não-zero) se **ocorrer algum erro**, caso **contrário**, retorna **falso** (zero).
- **Cada nova operação** com o arquivo **modifica a condição de erro**.

```
...  
FILE *arq;  
float valor;  
arq = fopen(..., "rb");  
...  
fread(&valor, ..., ..., arq);  
if(ferror(arq)) // se refere  
à leitura  
    printf("Erro na Leitura!");  
...  
...
```

Renomear ou Mover Arquivo

```
rename(nomeantigo, novonome)
```

- **Renomeia** um arquivo ou o **move** para um diretório destino.
- O **arquivo** deve estar **fechado**!
- Em caso de **sucesso**, retorna **zero**.

```
...  
    rename("a.dat", "b.dat");  
    /* a.dat -> b.dat */  
...
```

Remover Arquivo

```
remove(nomedoarquivo)
```

- **Apaga** o arquivo especificado do disco.
- O **arquivo** deve estar **fechado**!
- Retorna **zero** em caso de **sucesso**.

```
...  
    remove("tourstart.dat");  
    /*remove o arquivo  
    especificado*/  
...
```

Remover Arquivo

```
remove(nomedoarquivo)
```

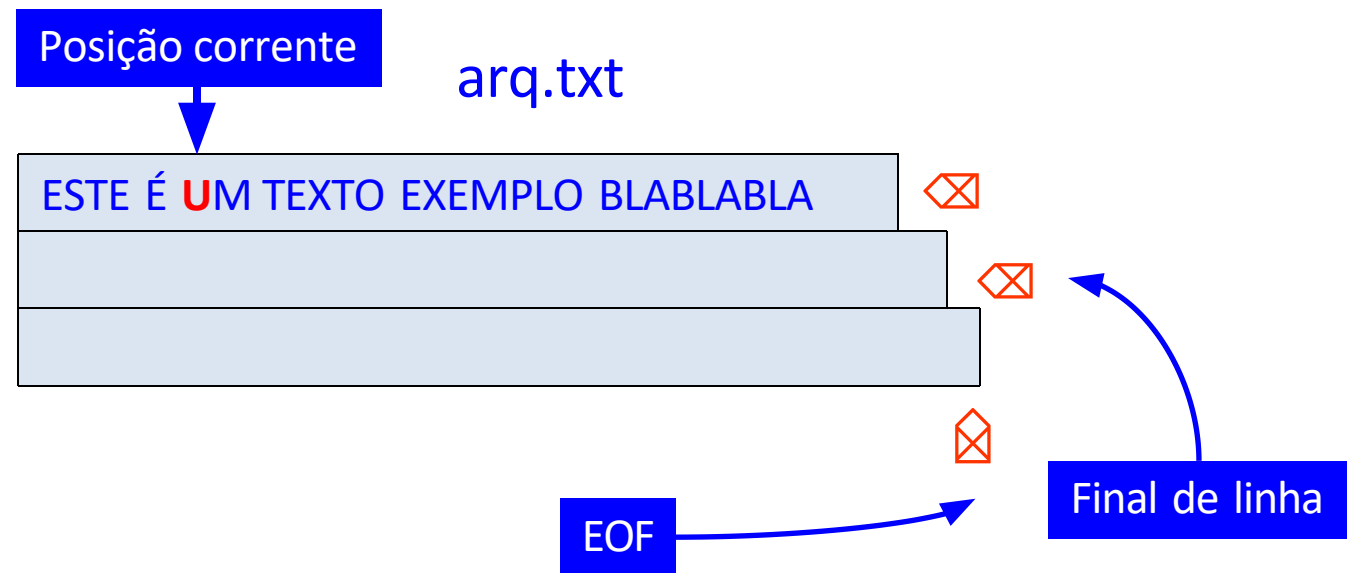
- **Apaga** o arquivo especificado do disco.
- O **arquivo** deve estar **fechado**!
- Retorna **zero** em caso de **sucesso**.

```
...  
    remove("tourstart.dat");  
    /*remove o arquivo  
    especificado*/  
...
```

Organização de Arquivos Texto

Arquivos texto são constituídos por uma sequência de caracteres, formando linhas de tamanho variável.

O **elemento corrente** de um **arquivo texto** é o **caractere** em que o sistema ficou posicionado após a **última operação** sobre o arquivo.



Modos de Abertura

Modo	Significado
"r"	Abre o arquivo para leitura, o arquivo deve existir ou um erro ocorre
"w"	Cria um arquivo vazio para escrita, caso um arquivo com o mesmo nome exista o seu conteúdo é apagado
"a"	Adiciona ao final de um arquivo (o arquivo é criado caso ele não exista)
"r+"	Abre um arquivo para leitura e escrita (o arquivo deve existir ou um erro ocorre)
"w+"	Cria um arquivo vazio para leitura e escrita (se um arquivo com o mesmo nome existe o conteúdo é apagado)
"a+"	Abre um arquivo para leitura e adição (todas as operações de escrita são feitas no final do arquivo, o arquivo é criado caso não exista)

Exemplo de Abertura

```
#include <stdio.h> // inclui as funções com arquivos

int main( )
{
    FILE *arq; // declaração de apontadores de arquivo
    arq = fopen("c:\\exemplo1.txt","r"); // abertura para leitura
    if (arq1 == NULL) // verifica se funcionou: retorno NULL indica erro
        printf("Erro na abertura do arquivo.");
    else{
        ....
    }
}
```

Atenção: dentro de programas, nomes de arquivos que incluam “\” devem ser escritos com “\\” quando informados em uma *string*.

A **leitura** ou **gravação** em um **arquivo texto** pode ser feita:

- **linha a linha** ou
- **caractere a caractere.**

Uma **linha** em um arquivo texto é o conjunto dos caracteres que se encontram no arquivo entre:

- O **início do arquivo** e a **primeira marca de fim de linha**;
- O **início do arquivo** e a **marca de fim de arquivo** (no caso de arquivos com uma só linha);
- Entre uma **marca de fim de linha** e a **marca de fim de arquivo**;
- Entre duas **marcas de fim de linha**.

Leitura de Texto por Linha

fscanf (<arquivo>, <formato>, &<variáveis>)

- Lê uma string/linha de um <arquivo>, baseado em <formato>, armazenando os valores em <variáveis>, semelhante ao **scanf**.
- Em leitura de **strings**, lê até encontrar **quebra de linha, espaço ou tabulação**. Cuidado ao ler **strings com espaços**.
- Caso seja usado para ler uma string, não inclui '\n' na string lida.
- **Retorna** o número de elementos lidos com sucesso (valor inteiro), ou -1 se nenhum elemento for lido.

fgets (<string>, <número>, <arquivo>)

- Lê uma string/linha de um arquivo, até encontrar '\n' ou EOF ou completar **número - 1** caracteres.
- Mantém na leitura o '\n' de fim da linha, se existir e couber, incluindo na string, e utiliza o último caractere reservado para o terminador de string.
- **Retorna** um ponteiro pra char (valendo NULL se EOF for lido antes de qualquer caractere)

Leitura de Caractere

`<caractere> = getc (<arquivo>)`

- **Retorna** um caractere, lido do arquivo.
- Se o final do arquivo for encontrado ou ocorrer erro, a função retorna **EOF**.

fprintf(<arquivo>, <formato>, <lista de variáveis>)

- **Escreve** uma string no <arquivo>, baseado em um <formato>, de maneira similar ao **printf()**.
- Para **quebra de linha** é necessária a inclusão de **'\n'** no formato do comando.
- Retorna o **número de caracteres escritos** (ou **número negativo** em caso de **erro**).

fputs (<string>, <arquivo>)

- **Escreve** o conteúdo da <string> no <arquivo>.
- Grava até atingir o **'\0'** mas **não grava ele** no arquivo, apenas **'\n'**, se existir.
- **Retorna** valor positivo (inteiro) em caso de sucesso ou EOF (macro definida na stdio, que é um inteiro negativo) em caso de erro.

fprintf(<arquivo>, <formato>, <lista de variáveis>)

- **Escreve** uma string no <arquivo>, baseado em um <formato>, de maneira similar ao **printf()**.
- Para **quebra de linha** é necessária a inclusão de **'\n'** no formato do comando.
- Retorna o **número de caracteres escritos** (ou **número negativo** em caso de **erro**).

fputs (<string>, <arquivo>)

- **Escreve** o conteúdo da <string> no <arquivo>.
- Grava até atingir o **'\0'** mas **não grava ele** no arquivo, apenas **'\n'**, se existir.
- **Retorna** valor positivo (inteiro) em caso de sucesso ou EOF (macro definida na stdio, que é um inteiro negativo) em caso de erro.

```
putc (<caractere>, <arquivo>)
```

- **Escreve** um <caractere> no <arquivo>.
- Para indicar **quebra de linha**, precisa gravar o caractere de quebra de linha ('\n').
- **Retorna** o código ASCII (inteiro) do caractere escrito em caso de sucesso, ou EOF em caso de erro.

Atenção!

- O **modo de abertura** do arquivo deve ser compatível com as **funções** que serão utilizadas sobre o arquivo.
- **Abrir** o arquivo para **escrita** e utilizar funções para **leitura** resultará em **falha**.
- Do mesmo modo, **abrir** o arquivo para **leitura** e utilizar funções de **escrita** resultará em **falha**.
- Lembre-se de que **cada função** tem seu **próprio modo de sinalizar** a ocorrência de falha.

Exemplo 1

Escreva um programa para salvar em um arquivo txt 5 números inteiros, cada um em uma linha. A seguir, o programa deve abrir, ler esses números e preencher um vetor de 5 posições.



Solução: fprintf e fscanf

```
// salva em arquivo texto NUMEL valores inteiros, um em cada
linha
// Lê do arquivo texto os valores salvos e armazena num vetor
#include <stdio.h>
#define NUMEL 5

int main( ){
    FILE *arq;
    char nome[16]; // para o nome do arquivo em disco
    int cont, valor, vetor[NUMEL];
    printf("Nome do arquivo: ");
    scanf("%s", nome);

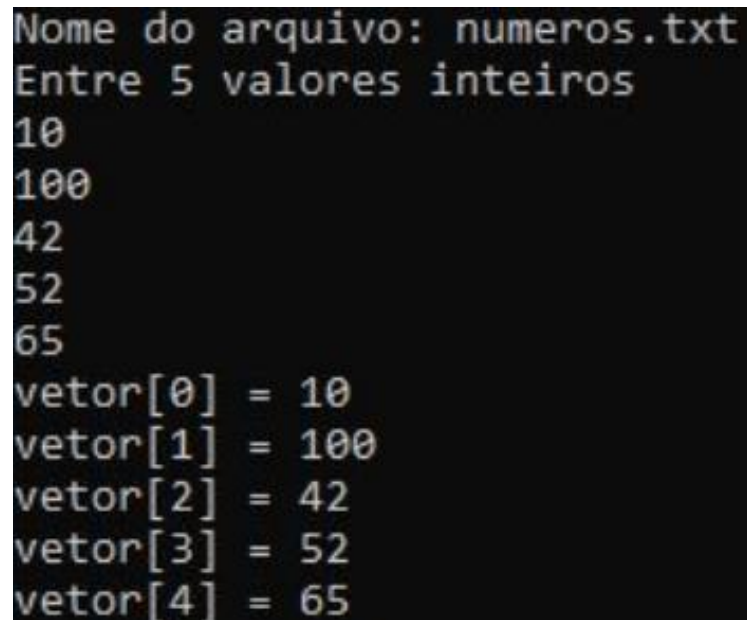
    if(!(arq = fopen(nome, "w"))) // abre para escrita modo texto
        printf("Erro na abertura\n");
    else{
        printf("Entre %d valores inteiros\n", NUMEL);
        for (cont = 0; cont < NUMEL; cont++){
            scanf("%d", &valor);
            fprintf(arq, "%d\n", valor);
        }
        fclose(arq);
    }
}
```

Solução: fprintf e fscanf

```
// abre o mesmo arquivo para leitura

if(!(arq = fopen(nome,"r"))) // abre para leitura
    printf("Erro na abertura\n");
else{
for (cont = 0; cont < NUMEL; cont++) // varre o arquivo
{
    fscanf(arq, "%d", &vetor[cont]);
    printf("vetor[%d] = %d\n", cont, vetor[cont]);
}
fclose(arq);
}

return 0;
}
```



```
Nome do arquivo: numeros.txt
Entre 5 valores inteiros
10
100
42
52
65
vetor[0] = 10
vetor[1] = 100
vetor[2] = 42
vetor[3] = 52
vetor[4] = 65
```

Exemplo 2

Escreva um programa que escreva em um arquivo todos os caracteres entre 'a' e 'e' e que depois lê o arquivo resultante, imprimindo os caracteres na tela.



Solução: getc e putc

```
#include <stdio.h>

int main( ){
    FILE *arq;
    char letra,ch;

    if(!(arq = fopen("Letras.txt","w+"))) // abre para escrita e leitura
        printf("Erro na abertura\n");
    else{ // grava caracteres 'a' à 'f' e insere fim de arquivo
        for(letra='a'; letra < 'f'; letra++)
            putc(letra,arq);

        rewind(arq);

        // lê arquivo texto até marca de fim do arquivo ou erro de leitura
        ch = getc(arq);

        while (ch != EOF){
            printf("%c ", ch); // imprime caractere na tela
            ch = getc(arq);
        }
        fclose(arq);
    }
    return 0;
}
```

Fim de Linha e Fim de Arquivo

- As **marcas de fim de linha** precisam ser **colocadas explicitamente** pelo programa.
- A **marca de fim de arquivo** é **gerada automaticamente**, quando o arquivo que foi **aberto com opção de escrita** é fechado (embora possa ser inserida pelo programa).
- A marca de final de linha em Linguagem C é o caractere `'\n'`, mas conforme o sistema operacional sua representação no arquivo pode mudar:
 - Em Unix, Linux, AIX, Xenix, Mac OS X, BeOS, etc:
LF (line feed) – `'\n'`
 - Em Commodore, Apple II e até Mac OS 9:
CR (carriage return) – `'\r'`
 - Em DOS, OS/2, Microsoft Windows:
CR+LF – `'\r'+'\\n'`
- Ou seja, cada sistema operacional converte o **`\n`** para o seu marcador específico de **fim de linha**!

Floats em Arquivos Texto

Geração de um arquivo texto com valores *float*, usando formato %6.2f (6 posições, no mínimo, com 2 casas decimais)

- A **gravação** do **arquivo texto** com *fprintf* segue as mesmas regras do *printf*.
- As casas **decimais obedecem** sempre o indicado no **formato**, mesmo que implique eliminação de casas ou reenchimento de zeros.
- O **número é armazenado na forma de uma sequência de caracteres** (representando dígitos, sinais e ponto decimal), sem nenhum **valor quantitativo** associado.
- Os caracteres utilizados são sempre ASCII, incluindo a representação ASCII do(s) caractere(s) branco(s) incluído(s), do ponto decimal e dos sinais + ou -, se existirem.

Exemplo 3

Considere um arquivo texto já existente “texto_input.txt”. Faça um programa que faça uma cópia desse arquivo texto para outro (copiando todo texto do primeiro arquivo para o segundo), chamado “texto_output.txt”



Solução: fgets e fputs

```
/*Gera uma copia de um arquivo texto*/
#include <stdio.h>
#define MAXLINHA 255
int main( )
{
    FILE *arq1; //entrada
    FILE *arq2; // saida
    char linha[MAXLINHA];

    if ((arq1 = fopen("texto_input.txt", "r")) == NULL)
        printf("Erro ao abrir arquivo entrada \n");
    else
        if ((arq2 = fopen("texto_output.txt", "w")) == NULL)
            printf("Erro ao abrir arquivo saida \n");
        else
        {
            while (!feof(arq1))
                if (fgets(linha, sizeof(linha), arq1) != NULL )
                { // leu linha
                    fputs(linha, arq2); // grava linha
                }
            fclose(arq1);
            fclose(arq2);
        }
    return 0;
}
```

Funciona mesmo se o arquivo tiver uma única linha com um número de caracteres maior que MAXLINHA.

Exemplo 4

Faça um programa para substituir um determinado caractere em um arquivo texto existente por outro (ambos obtidos por leitura via teclado). As alterações devem ser realizadas diretamente sobre o arquivo. Ou seja, sem armazenar todo o conteúdo lido em uma string, operar a substituição na string e gravar a string resultante.

Texto do arquivo: o rato roeu a roupa do rei de Roma

Substituir 'o' por '@'

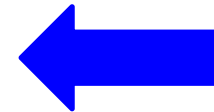
Numero de caracteres alterados: 6

Novo texto do arquivo: @ rat@ r@eu a r@upa d@ rei de R@ma



Solução: fseek, putc e fflush

```
#include<stdio.h>
int main( ){
    FILE *arq;
    char antigo, novo, temp, caract;
    int alt = 0;
    if ((arq = fopen("texto_original.txt", "r+")) == NULL)
        printf("Erro ao abrir \n");
    else
    {
        printf("\nCaractere a procurar: \n");
        scanf(" %c", &antigo);
        printf("\nSubstituir por: \n");
        scanf(" %c", &novo);
        while (!feof(arq))
        {
            temp = getc(arq);
            if(temp == antigo){
                alt++; // conta quantos foram alterados
                /*volta a posição do caractere procurado*/
                fseek(arq, -sizeof(char), SEEK_CUR);
                putc(novo, arq); // substitui
                fflush(arq);
            }
        }
        fclose(arq);
        printf("\nNumero de caracteres alterados: %d\n", alt);
    }
    return 0;
}
```



Troca de um
caractere por
outro